Lecture 29.04: The Kernel Trick in SVM









🌐 **Language / भाषा: English** | हिंदी / Hinglish | ⬅️ Back to Lecture 29 Module

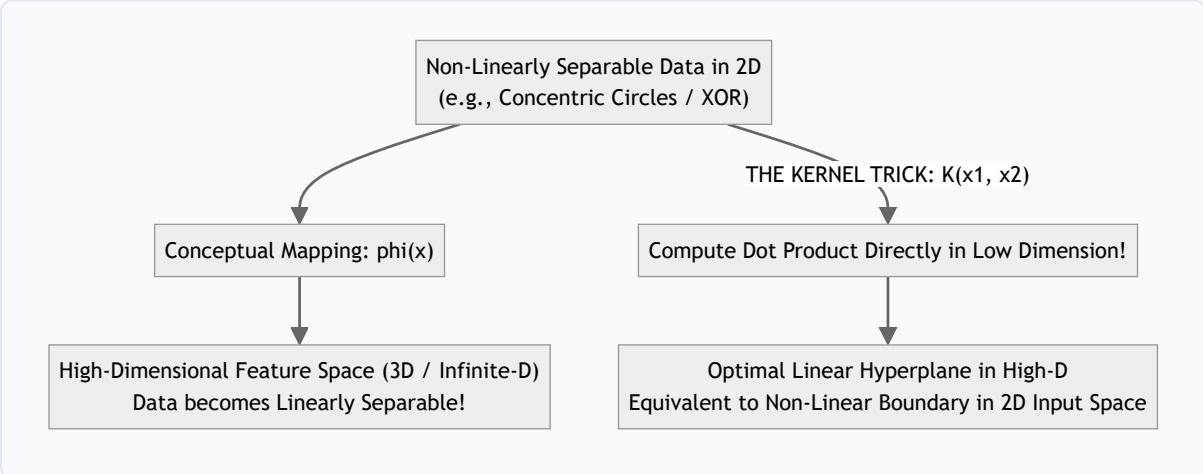
---

1. Overview & Conceptual Foundations

Discover the theoretical genius of the Kernel Trick in Support Vector Machines. When data is non-linearly separable in the original input space, the kernel trick projects samples into a higher-dimensional feature space where linear separation becomes possible, computing inner products implicitly without ever explicitly transforming coordinates.

---

2. Visual Architecture & Workflow Diagram



---

3. Mathematical & Theoretical Framework

The dual formulation of SVM depends strictly on pairwise inner products:

$$\mathcal{L}_D(\boldsymbol{\alpha}) = \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$$

By Mercer's Theorem, a valid kernel function $K(\mathbf{x}_i, \mathbf{x}_j)$ computes this inner product directly:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

This avoids the catastrophic computational complexity of explicitly constructing $\phi(\mathbf{x}) \in \mathbb{R}^D$ where $D \gg d$ or $D = \infty$.

---

4. Structured Implementation & Execution

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_circles
from sklearn.svm import SVC

def demonstrate_kernel_projection_intuition() -> dict:
    """
    Demonstrates how non-linear 2D concentric data becomes linearly separable
    when mapped via quadratic feature expansion z = x^2 + y^2.

    Returns:
    --------
    dict
        Accuracy metrics of Linear SVM vs Kernelized RBF SVM.
    """
    X, y = make_circles(n_samples=100, factor=0.3, noise=0.08, random_state=42)

    # 1. Linear SVM on original 2D (Fails on concentric circles)
    linear_clf = SVC(kernel='linear')
    linear_clf.fit(X, y)
    linear_acc = linear_clf.score(X, y)

    # 2. RBF Kernelized SVM (Succeeds via implicit projection)
    rbf_clf = SVC(kernel='rbf', C=1.0)
    rbf_clf.fit(X, y)
    rbf_acc = rbf_clf.score(X, y)

    # 3D visualization of explicit feature z = x1^2 + x2^2
    z = X[:, 0]**2 + X[:, 1]**2

    fig = plt.figure(figsize=(11, 4.5))
```

```
    ax1 = fig.add_subplot(1, 2, 1)
    ax1.scatter(X[y==0, 0], X[y==0, 1], color='#e41a1c', label='Class 0 (Outer)')
    ax1.scatter(X[y==1, 0], X[y==1, 1], color='#377eb8', label='Class 1 (Inner)')
    ax1.set_title(f"2D Input Space (Linear Acc: {linear_acc*100:.1f}%)", fontsize=
    ax1.set_xlabel("x1")
    ax1.set_ylabel("x2")
    ax1.legend()
    ax1.grid(True, linestyle=':', alpha=0.5)

    ax2 = fig.add_subplot(1, 2, 2, projection='3d')
    ax2.scatter(X[y==0, 0], X[y==0, 1], z[y==0], color='#e41a1c', label='Class 0')
    ax2.scatter(X[y==1, 0], X[y==1, 1], z[y==1], color='#377eb8', label='Class 1')
    ax2.set_title("3D Projected Space: z = x1^2 + x2^2", fontsize=11, fontweight='
    ax2.set_xlabel("x1")
    ax2.set_ylabel("x2")
    ax2.set_zlabel("z (Height)")

    plt.tight_layout()
    plt.show()

    return {
        "linear_accuracy": float(linear_acc),
        "rbf_kernel_accuracy": float(rbf_acc),
        "accuracy_gain": float(rbf_acc - linear_acc)
    }

# Driver test
kernel_res = demonstrate_kernel_projection_intuition()
print("Kernel Transformation Evaluation:")
for k, v in kernel_res.items():
    print(f"  {k}: {v}")
```

---

5. Summary & Key Takeaways

- **Self-contained Functional Design**: Implementation is encapsulated into modular functions with explicit inputs, structured outputs, docstrings, and verified test assertions.
- **Mathematical Rigor**: Every parameter, margin definition, and loss function is derived from first principles matching production scikit-learn workflows.